JDBC

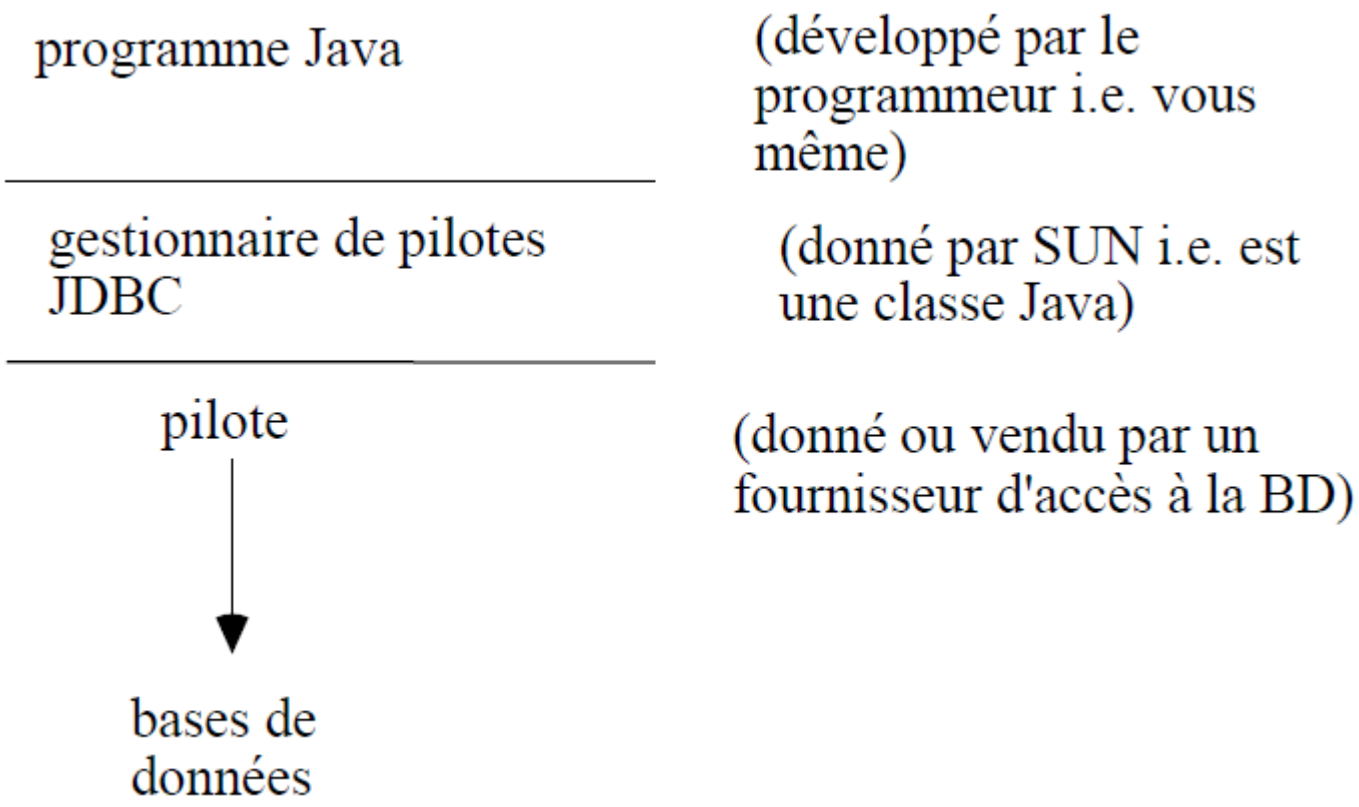
JDBC

- Java DataBase Connectivity
- API Java permettant
  - la connexion avec les bases de données (e.g. SQLite, MySQL)
  - l'envoi de requête
  - le traitement des résultats

- **Permet de se connecter de la même manière avec la plupart des systèmes de Gestion de Base de Données.**

- Une des grosses difficultés tient plus à la préparation de la base de données pour être utilisée avec JDBC

ARCHITECTURE DE JDBC



PRÉPARATION DE L'ENVIRONNEMENT

- Vous avez besoin d'un connector !
  - Soit télécharger un .jar et l'ajouter dans le projet Java (classpath)
  - Soit utiliser MAVEN : ajouter la bonne dependence dans le pom.xml

- Exemple, pour SQLite :

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
</dependency>
```

EXEMPLE 0 : SE CONNECTER À UNE BASE DE DONNÉES

4 ÉTAPES

1. Créer un objet **Connection** qui représente la connexion au serveur de BD
2. Créer un ou plusieurs objets **Statement** pour manipuler la BD
3. Utiliser les objets **Statement** pour manipuler la BD
4. Fermer la connexion à la BD

1. Créer un objet Connection qui représente la connexion au serveur de BD

- **L'URL du serveur de BD**

```
String url = "jdbc:sqlite:./db/conferences.db";
```

```
String urlMySQL = "jdbc:mysql://localhost:3306/mysql";
```

- **Créer une connexion à la BD**

```
Connection con = DriverManager.getConnection(url);
```

```
Connection conMySQL = DriverManager.getConnection(urlMySQL,"root", "");
```

  - Si la tentative échoue, une exception de type **SQLException** sera levée. L'information pertinente à la nature du problème sera encapsulée dans l'objet SQLException

2.CRÉER UN OBJET STATEMENT

- Un objet **Connection** définit une connexion (session) avec un BD spécifique.

- Des instructions (statement) SQL sont exécutées et des résultats sont retournés dans le contexte d'une connexion.

- Un objet **Statement** est utilisé pour exécuter une/plusieurs instructions SQL et pour retourner les résultats qu'elle(s) produi(sen)t

```
Statement stmt;
stmt = con.createStatement();
```

3. MÉTHODES DE STATEMENT

- **Statement** possède de nombreuses méthodes qui permettent d'accéder à la BD et de manipuler ses données

- **`int executeUpdate(String SQLcomm)`**
  SQLcomm doit être une commande SQL valide
  - Cette méthode est utilisée pour exécuter les instructions SQL INSERT, UPDATE ou DELETE
  - Les autres instructions SQL qui ne retournent rien peuvent être exécutées en utilisant cette méthode

- **`ResultSet executeQuery(String sql)`**
  - Exécute une requête SQL sql qui renvoie un résultat sous forme de ResultSet.
  - Utilisé pour les instructions de type SELECT

4.FERMER LA CONNEXION

```
con.close();
```

RÉSUMÉ

```
import java.sql.Connection;
import java.sql.Statement;

public class prog1 {

  public static void main(String args[]){
   try {
      Statement stmt;
      String url = "jdbc:sqlite:./db/santa.db";
      Connection con = DriverManager.getConnection(url);
      stmt = con.createStatement();
      // ……————————————.
      con.close();
   } catch( Exception e ) {  e.printStackTrace(); }//end catch
  }//end main
}
```

RÉSUMÉ (VERSION AVEC GESTION DES RESSOURCES DANS LE TRY)

```
import java.sql.Connection;
import java.sql.Statement;

public class prog1 {

  public static void main(String args[]){
   try {
     String url = "jdbc:sqlite:./db/santa.db";
     try (Connection conn = DriverManager.getConnection(url)) {
       System.out.println("Connection to SQLite has been established. »);
       Statement stmt;
       stmt = con.createStatement();
      // ……————————
    } catch (SQLException e){  e.printStackTrace(); }//end catch
  }//end main
}
```

EXEMPLES

Manipuler la base et ses tables

CRÉATION DE LA TABLE JOUETS

```
stmt.executeUpdate(
"CREATE TABLE IF NOT EXISTS Jouets(
        jouet_id INTEGER PRIMARY KEY AUTOINCREMENT,
        jouet_nom text NOT NULL);");
```

- La nouvelle table aura deux colonnes
  - **jouet_id** : un entier
  - **jouet_nom** : une chaîne de 15 caractères max

INSÉRER DES VALEURS DANS LA TABLE

- **jouet_id** est auto-incrémenté, donc pas besoin de le préciser
- « Playmobil » est inséré dans la colonne **jouet_nom**

```
stmt.executeUpdate("INSERT INTO Jouets(jouet_nom) VALUES('Playmobil')");
```

INSERTION DE 4 LIGNES SUPPLÉMENTAIRES

```
stmt.executeUpdate("INSERT INTO Jouets(jouet_nom) VALUES(’Velo')");

stmt.executeUpdate("INSERT INTO Jouets(jouet_nom) VALUES(’Poupee')");

stmt.executeUpdate("INSERT INTO Jouets(jouet_nom) VALUES(’Orange')");

stmt.executeUpdate("INSERT INTO Jouets(jouet_nom) VALUES(’Balai')");
```

INSERTION DE 4 LIGNES SUPPLÉMENTAIRES

- On peut aussi ajouter des requêtes incomplètes (ou paramètrées) avec **PreparedStatement**.

```
List<String> jouets = List.of("Velo","Poupee","Orange","Balai");

 String sql = "INSERT INTO Jouets(jouet_nom) VALUES(?)";
 PreparedStatement pstmt = conn.prepareStatement(sql);

 for(String j : jouets){
    pstmt.setString(1,j);
    pstmt.executeUpdate();
 }
```

INTERROGATION DE LA BASE

- Jusqu'à présent nous n'avons utilisé que la méthode **executeUpdate** de l'interface **Statement**

- Rappel : executeUpdate peut être utilisée pour exécuter des commandes SQL qui ne retournent aucun résultat

- Pour exécuter une commande SQL qui retourne des résultats nous utiliserons la méthode **executeQuery** sur l'objet **Statement**

RAPPEL : MÉTHODES DE STATEMENT

- **ResultSet executeQuery(String sql)**
  - Exécute une requête SQL sql qui renvoie un résultat sous forme de **ResultSet**.

- Exception : SQLException
  Si :
  - l'acces à la base de données provoque une erreur
  - Si executeQuery est appelé sur une Statement fermée
  - Si la requête produit autre chose qu'un ResultSet

UN OBJET RESULTSET

- Un objet ResultSet fourni un accès à une table de données encapsulée
- L' object maintient un curseur pointant sur la ligne de données courante
- Initialement le curseur est positionné **avant** la première ligne

- La méthode **next()** de ResultSet bouge le curseur à la ligne suivante (similaire à un itérateur ou un énumérateur en Java)
- La méthode next() renvoie false s'il n'y a pas de ligne suivante.

MANIPULATION D'UN RESULTSET

```
stmt = con.createStatement();

ResultSet rs = stmt.executeQuery(
                    "SELECT * FROM Jouets ORDER BY jouet_id");

while(rs.next()){

}//end while
```

MÉTHODES D'UN OBJET RESULTSET

- Une fois que les résultats sont encapsulés dans un objet ResultSet, l'interface ResultSet fournit plusieurs méthodes qui permettent d'extraire les informations de l'objet

- Les méthodes *get** (comme le *getString*) retourne les valeurs des colonnes pour la ligne courante :
  - `getString(String columnLabel)`
  - `getString(int columnIndex)`

- On peut retourner les valeurs soit en utilisant l'indice de la colonne soit son nom.
- Note: les colonnes sont numérotées **à partir de 1 pas de 0**
- Le pilote JDBC gère en principe la conversion des données en type Java

EXEMPLE

```
stmt = con.createStatement();

ResultSet rs = stmt.executeQuery("SELECT * FROM Jouets ORDER BY jouet_id");

System.out.println("Liste des cadeaux :");

while(rs.next()){
   int monJouetId= rs.getInt("jouet_id");
   String nomJouet = rs.getString("jouet_nom");
   System.out.println("\tid= " + monJouetId + "\tnom = " + nomJouet);
}//end while
```

Une SQLException est levée par get****() si :

- columnIndex n'est pas valide
- Erreur d'accès à la BD ou ResultSet fermé

SORTIE

Liste des cadeaux:

id= 1 nom = Playmobil

id= 2 nom = Velo

id= 3 nom = Poupee

id= 4 nom = Orange

id= 5 nom = Balai

LA CLASSE RESULTSETMETADATA

- Un objet de type ResultSetMetaData permet d'avoir des information sur le type et les propriétés des colonnes d'un objet ResultSet.

```
String query = "SELECT * FROM Jouets";
ResultSet results = stmt.executeQuery(query);

ResultSetMetaData rsmd = results.getMetaData();
int numcols = rsmd.getColumnCount();

for(int i = 1;i<=numcols;i++){
    System.out.println(rsmd.getColumnName(i));
 }
```

AFFICHER LES DONNÉES DE LA LIGNE 2

Affichage de la ligne 2 :

id= 2 nom = Velo

```
System.out.println("Affichage de la ligne 2 :");
if( rs.absolute(2) ){
  int monJouetId= rs.getInt("jouet_id");
  String nomJouet = rs.getString("jouet_nom");
  System.out.println("\tid= " + monJouetId  + "\tnom = " +
nomJouet);
}//end if
```